

UNIVERSIDAD PERUANA LOS ANDES
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y
COMPUTACIÓN



Base de Datos II

Semana 13 Practica

AUTOR

Vicente Ramos

Elían

DOCENTE

RAUL FERNANDEZ, Bejarano

CICLO

V

HUANCAYO – PERÚ – 2025

Practica-semana 13

Ejercicio 1 Código de Prueba Ejecuta este script para simular una consulta lenta. Como le hemos puesto un retraso de 2 segundos, el Extended Event debería capturarla.

--problema 01

```
USE master;
GO
```

```
-- 1. Verificar si la sesión ya existe y eliminarla
IF EXISTS (SELECT * FROM sys.server_event_sessions WHERE name =
'CapturaConsultasLentas_QhatuPeru')
    DROP EVENT SESSION [CapturaConsultasLentas_QhatuPeru] ON SERVER;
GO
```

```
-- 2. Crear la Sesión de Extended Events
CREATE EVENT SESSION [CapturaConsultasLentas_QhatuPeru] ON SERVER
ADD EVENT sqlserver.sql_statement_completed
(
    ACTION
    (
        sqlserver.database_name,    -- BD donde ocurrió
        sqlserver.sql_text,        -- Query ejecutado
        sqlserver.username,        -- Usuario
        sqlserver.client_app_name  -- Aplicación
    )
    WHERE
    (
        -- Duración mayor a 1 segundo (1 millón de microsegundos)
        [duration] > 1000000
        AND
        -- Filtro por nombre de base de datos
        [sqlserver].[database_name] = N'QhatuPeru'
    )
)
ADD TARGET package0.event_file
(
    -- ⚠ CORRECCIÓN: Solo ponemos el nombre del archivo.
    -- SQL Server lo guardará en su carpeta de LOGS predeterminada
    automáticamente.
    SET filename = N'ConsultasLentas_QhatuPeru.xel',
        max_file_size = 5,
        max_rollover_files = 2
)
WITH
(
    MAX_MEMORY = 4096 KB,
    EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS,
    MAX_DISPATCH_LATENCY = 30 SECONDS,
    STARTUP_STATE = OFF
);
GO
```

```
-- 3. Iniciar la sesión
ALTER EVENT SESSION [CapturaConsultasLentas_QhatuPeru] ON SERVER STATE = START;
GO
```

```

USE QhatuPeru;
GO

-- =====
-- PRUEBA: Simulación de consulta lenta
-- =====

-- 1. Esta consulta es RÁPIDA (No se guardará)
SELECT TOP 10 * FROM dbo.ARTICULO;

-- 2. Esta consulta es LENTA (Espera 2 seg y luego ejecuta)
-- Como 2 seg > 1 seg, esta SÍ quedará registrada en el archivo .xel
WAITFOR DELAY '00:00:02';
SELECT * FROM dbo.PROVEEDOR;
GO

USE master;
GO

-- =====
-- CONSULTA: Leer los datos capturados en el archivo .xel
-- =====

-- 1. Declaramos variables para encontrar la ruta del archivo automáticamente
DECLARE @RutaArchivo NVARCHAR(256);
DECLARE @RutaConComodin NVARCHAR(256);

-- 2. Obtenemos la ruta exacta donde SQL guardó el archivo
SELECT @RutaArchivo = CAST(target_data AS
XML).value('(/EventFileTarget/File/@name)[1]', 'VARCHAR(MAX)')
FROM sys.dm_xe_session_targets
WHERE target_name = 'event_file'
AND event_session_address = (SELECT address FROM sys.dm_xe_sessions WHERE name =
'CapturaConsultasLentas_QhatuPeru');

-- 3. Preparamos la ruta para leer (Agregamos *.xel para leer todos los archivos
generados)
-- (Truco: Quitamos la extensión .xel exacta y ponemos *.xel para leer la
secuencia completa)
IF @RutaArchivo IS NOT NULL
BEGIN
    SET @RutaConComodin = LEFT(@RutaArchivo, LEN(@RutaArchivo) - 4) + '*.xel';

    PRINT 'Leyendo archivo desde: ' + @RutaConComodin;

-- 4. CONSULTA FINAL: Leemos el XML y lo mostramos como tabla ordenada
SELECT
    object_name AS Evento,
    DATEADD(HOUR, -5, timestamp_utc) AS FechaHora_Peru, -- Ajuste horario
aprox (UTC-5)
    payload.value('(action[@name="database_name"]/value)[1]',
'NVARCHAR(128)') AS BaseDeDatos,
    payload.value('(action[@name="username"]/value)[1]', 'NVARCHAR(128)') AS
Usuario,
    payload.value('(action[@name="client_app_name"]/value)[1]',
'NVARCHAR(128)') AS Aplicacion,

```

```

        payload.value('(<data[@name="duration"]>/value)[1]', 'BIGINT') / 1000000.0
AS Duracion_Segundos,
        payload.value('(<action[@name="sql_text"]>/value)[1]', 'NVARCHAR(MAX)') AS
Query_Ejecutado
FROM
(
    SELECT
        object_name,
        timestamp_utc,
        CAST(event_data AS XML) AS payload
    FROM sys.fn_xe_file_target_read_file(@RutaConComodin, null, null,
null)
) AS Datos
ORDER BY timestamp_utc DESC;
END
ELSE
BEGIN
    PRINT 'Error: No se encontró la sesión activa o el archivo. Asegúrate de que
la sesión esté en estado START.';
END
GO

```

CodArticulo	CodLinea	CodProveedor	DescripcionArticulo	Presentacion	PrecioProveedor	StockActual	StockMinimo	Descontinuado
1	1	1	Traje Iron Man	Unidad	500000.00	1	1	0
2	2	1	Caja Explosivos	Caja	150.00	50	10	0
3	3	3	Sable Luz	Unidad	5000.00	10	2	0
4	4	4	Bataram	Docena	450.00	20	5	0
5	5	20	Condensador	Unidad	1955.00	5	1	0
6	6	8	Pan Lembas	Paq	5.00	500	100	0
7	7	4	Varita Sauco	Unidad	8000.00	3	1	0
8	8	12	Pokebola	Caja	200.00	150	20	0

CodProveedor	NomProveedor	Representante	Direccion	Ciudad	Departamento	CodigoPostal	Telefono	Fax
1	Stark Industries	Tony Stark	Torre Avengers	NY	NY	10001	555-IRON	555-01
2	Acme Corp	Wile Coyote	Desierto	AZ	AZ	85001	555-BEEP	555-02
3	Wayne Enter...	Lucius Fox	Gotham Center	NJ	NJ	07001	555-BATS	555-03
4	Umbrella Corp	Weaker	Raccoon City	MW	MW	66666	555-VIRU	555-04
5	Cyberdyne	Skyнет	Silicon Valley	CA	CA	94016	555-TE...	555-05
6	Oscorp	Osborn	Queens	NY	NY	11101	555-GOBL	555-06
7	Olivanders	Garrick	Diagon Alley	UK	LDN	WC2H	555-WA...	555-07
8	Capsule Corp	Bulma	West City	DBZ	WC	90210	555-GO...	555-08

Evento	FechaHora_Peru	BaseDeDatos	Usuario	Aplicacion	Duracion_Segundos	Query_Ejecutado
sql_statement_completed	2025-12-03 11:21:49.0570000	NULL	NULL	NULL	NULL	NULL

1. *Script de Prueba: Usa WAITFOR DELAY '00:00:02' para pausar el sistema intencionalmente por 2 segundos. Esto obliga a que la consulta supere el límite de 1 segundo (1,000,000 microsegundos) que configuramos en la sesión.*
2. *Script de Lectura:*
 - *Primero busca automáticamente en sys.dm_xe_session_targets dónde guardó SQL Server el archivo.*
 - *Luego usa la función sys.fn_xe_file_target_read_file para abrir ese archivo físico.*
 - *Finalmente, convierte el formato XML (que es difícil de leer) en una tabla limpia con columnas como Usuario, Duracion_Segundos y Query_Ejecutado.*

Ejercicio 2 Crear índices para mejorar la búsqueda de clientes

En la tabla Clientes, mejorar el rendimiento de búsqueda por DNI y Apellidos creando índices adecuados.

1. *Script de la solución en T-SQL SQL*

--problema 02

```
USE QhatuPeru;
GO

-- =====
-- 1. PRE-REQUISITO: Crear la tabla CLIENTE (Ya que no existe)
-- =====
IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'CLIENTE')
BEGIN
    CREATE TABLE dbo.CLIENTE (
        CodCliente INT IDENTITY(1,1) PRIMARY KEY, -- Esto crea el índice
        Clustered automáticamente
        DNI CHAR(8) NOT NULL,
        Apellidos VARCHAR(50) NOT NULL,
        Nombres VARCHAR(50) NOT NULL,
        Direccion VARCHAR(100),
        Email VARCHAR(100),
        Telefono VARCHAR(15)
    );
    PRINT 'Tabla CLIENTE creada correctamente.';

    -- Insertamos algunos datos de prueba
    INSERT INTO dbo.CLIENTE (DNI, Apellidos, Nombres, Email) VALUES
    ('12345678', 'Perez', 'Juan', 'juan@mail.com'),
    ('87654321', 'Gomez', 'Maria', 'maria@mail.com'),
    ('11223344', 'Diaz', 'Carlos', 'carlos@mail.com');
END
GO

-- =====
-- 2. SOLUCIÓN: Creación de Índices para optimizar búsquedas
-- =====

-- A) Índice para búsqueda por DNI
-- Justificación: El DNI es único, por lo que usamos UNIQUE para máxima
-- velocidad.
IF NOT EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Cliente_DNI')
BEGIN
    CREATE UNIQUE NONCLUSTERED INDEX IX_Cliente_DNI
    ON dbo.CLIENTE(DNI);
    PRINT 'Índice IX_Cliente_DNI creado.';
END
GO

-- B) Índice para búsqueda por Apellidos
-- Justificación: Los apellidos pueden repetirse, usamos un índice estándar (Non-
-- Clustered).
-- Incluimos 'Nombres' como columna incluida para evitar ir a la tabla principal
-- si solo pedimos nombres.
IF NOT EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Cliente_Apellidos')
BEGIN
    CREATE NONCLUSTERED INDEX IX_Cliente_Apellidos
    ON dbo.CLIENTE(Apellidos)
    INCLUDE (Nombres); -- INCLUDE: Optimización extra (Covering Index)
    PRINT 'Índice IX_Cliente_Apellidos creado.';
END
GO
```

GO

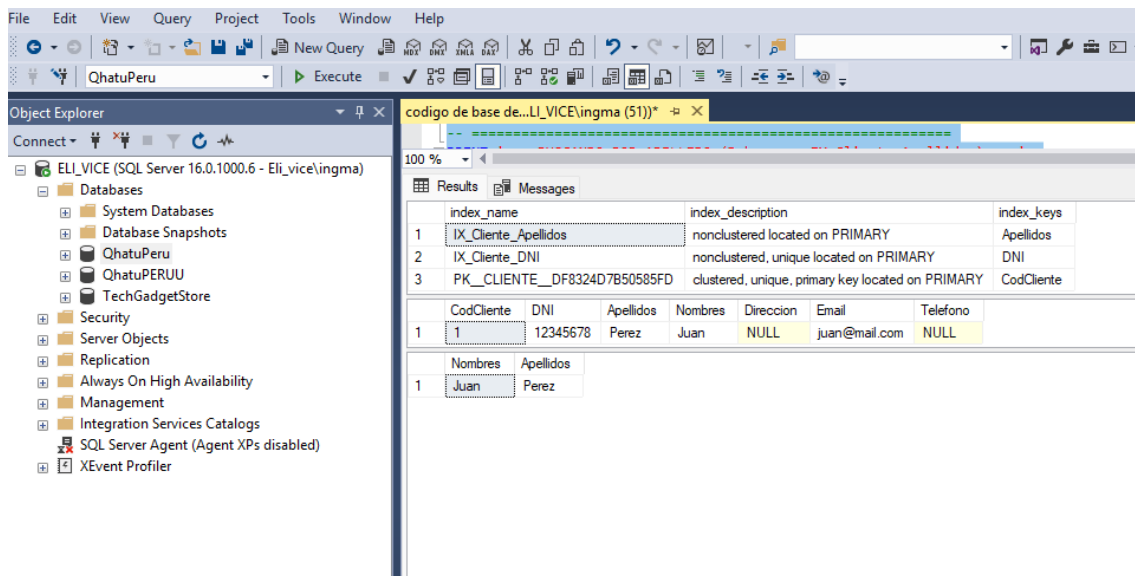
-- consultas

USE QhatuPeru;
GO

```
-- =====  
-- 1. VERIFICACIÓN FÍSICA: ¿Existen los índices?  
-- =====  
-- Este comando del sistema te lista todos los índices de la tabla  
PRINT '--- LISTA DE ÍNDICES EN TABLA CLIENTE ---';  
EXEC sp_helpindex 'dbo.CLIENTE';  
GO
```

```
-- =====  
-- 2. PRUEBA DE RENDIMIENTO: Usando el Índice de DNI  
-- =====  
PRINT '--- BUSCANDO POR DNI (Debe usar IX_Cliente_DNI) ---';  
  
-- Esta consulta buscará directamente el DNI sin leer toda la tabla.  
-- Al ser UNIQUE, SQL Server sabe que solo hay uno y se detiene al encontrarlo.  
SELECT * FROM dbo.CLIENTE  
WHERE DNI = '12345678';  
GO
```

```
-- =====  
-- 3. PRUEBA DE RENDIMIENTO: Usando el Índice de Apellidos  
-- =====  
PRINT '--- BUSCANDO POR APELLIDO (Debe usar IX_Cliente_Apellidos) ---';  
  
-- Esta consulta es especial: Solo pedimos 'Nombres' y filtramos por 'Apellidos'.  
-- Como incluimos 'Nombres' en el índice (INCLUDE), SQL ni siquiera mirará la  
tabla principal.  
-- Esto se llama "Index Seek" (Búsqueda en índice).  
SELECT Nombres, Apellidos  
FROM dbo.CLIENTE  
WHERE Apellidos = 'Perez';  
GO
```



Índice UNIQUE en DNI (IX_Cliente_DNI):

o Se eligió un índice No Agrupado Único (Unique Non-Clustered). o ¿Por qué Único? El DNI es un valor que lógicamente no debe repetirse. Al marcarlo como UNIQUE, SQL Server optimiza la búsqueda sabiendo que, una vez que encuentra el valor, no necesita seguir buscando más filas. Esto convierte una búsqueda completa (Scan) en una búsqueda puntual (Seek) extremadamente rápida.

- Índice Estándar en Apellidos (IX_Cliente_Apellidos):
 - Se creó un índice No Agrupado (Non-Clustered).
 - Estructura de Árbol-B: Al crear este índice, SQL Server crea una estructura separada ordenada alfabéticamente por Apellido. Sin este índice, si buscas WHERE Apellidos = 'Gomez', SQL tendría que leer toda la tabla fila por fila (Table Scan). Con el índice, salta directamente a la letra "G".
 - Cláusula INCLUDE: Se agregó INCLUDE (Nombres). Esto permite que si haces una consulta SELECT Nombres FROM CLIENTE WHERE Apellidos = 'X', SQL obtenga el dato directamente del índice sin tener que ir a buscar a la tabla principal (Lookup), reduciendo las operaciones de lectura en disco.

4. Explicación de las buenas prácticas utilizadas en el proyecto

1. Convención de Nombres (IX_Tabla_Columna): Se utilizó el prefijo IX_ seguido del nombre de la tabla y la columna afectada. Esto permite identificar rápidamente en el explorador de objetos qué índices existen y a qué columnas pertenecen, facilitando el mantenimiento.
2. Selectividad de Columnas: Se crearon índices solo en columnas que se usan frecuentemente en cláusulas WHERE (DNI y Apellidos). No se indexaron columnas como Direccion porque raramente se busca por dirección exacta, y indexar todo innecesariamente ralentiza las inserciones (INSERT) y actualizaciones (UPDATE).
3. Uso de UNIQUE para integridad de datos: Además de mejorar el rendimiento, el índice en DNI actúa como una restricción ("constraint"), impidiendo por diseño que se inserten dos clientes con el mismo documento, lo cual garantiza la calidad de los datos.

Problema 3 Crear índices para mejorar la búsqueda de clientes

1. Enunciado del ejercicio

En la tabla `CLIENTE`, mejorar el rendimiento de las consultas que buscan por **DNI** y por **Apellidos** creando los índices adecuados para convertir escaneos de tabla (lentos) en búsquedas indexadas (rápidas).

2. Script de la solución en T-SQL

-- problema 03

```
USE QhatuPeru;
GO
```

```
-- =====
-- A. PREPARACIÓN: Crear Tabla y Datos (Porque no la tienes)
-- =====
```

```
IF NOT EXISTS (SELECT * FROM sys.tables WHERE name = 'CLIENTE')
BEGIN
```

```
    CREATE TABLE dbo.CLIENTE (
        CodCliente INT IDENTITY(1,1) PRIMARY KEY,
        DNI CHAR(8) NOT NULL,
        Apellidos VARCHAR(50) NOT NULL,
        Nombres VARCHAR(50) NOT NULL,
        Direccion VARCHAR(100),
        Email VARCHAR(100),
        Telefono VARCHAR(15)
    );
```

```
-- Insertamos datos de prueba para que los índices tengan algo que indexar
INSERT INTO dbo.CLIENTE (DNI, Apellidos, Nombres, Email) VALUES
('10000001', 'Perez', 'Juan', 'juan@mail.com'),
('20000002', 'Gomez', 'Maria', 'maria@mail.com'),
('30000003', 'Quispe', 'Carlos', 'carlos@mail.com'),
('40000004', 'Flores', 'Ana', 'ana@mail.com'),
('50000005', 'Diaz', 'Pedro', 'pedro@mail.com');
```

```
PRINT 'Tabla CLIENTE creada y poblada correctamente.';
```

```
END
GO
```

```
-- =====
-- B. SOLUCIÓN: Creación de Índices
-- =====
```

```
-- 1. Índice para búsqueda por DNI
```

```
-- Tipo: UNIQUE NONCLUSTERED (Porque el DNI no se repite)
```

```
IF NOT EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Cliente_DNI')
BEGIN
```

```
    CREATE UNIQUE NONCLUSTERED INDEX IX_Cliente_DNI
    ON dbo.CLIENTE(DNI);
```

```
PRINT 'Índice IX_Cliente_DNI (Único) creado exitosamente.';
```

```
END
GO
```

```
-- 2. Índice para búsqueda por Apellidos
```

```
-- Tipo: NONCLUSTERED con INCLUDE (Optimizado para búsquedas de nombres)
```

```
IF NOT EXISTS (SELECT * FROM sys.indexes WHERE name = 'IX_Cliente_Apellidos')
BEGIN
```



```

CREATE NONCLUSTERED INDEX IX_Cliente_Apellidos
ON dbo.CLIENTE(Apellidos)
INCLUDE (Nombres); -- Truco de rendimiento: Incluimos el nombre aquí mismo

PRINT 'Índice IX_Cliente_Apellidos creado exitosamente.';
END
GO

-- consultas

USE QhatuPeru;
GO

-- =====
-- 1. VERIFICACIÓN TÉCNICA: ¿Los índices existen?
-- =====
PRINT '--- VERIFICANDO ÍNDICES EN TABLA CLIENTE ---';

-- Esta consulta del sistema te muestra qué índices tiene la tabla
SELECT
    name AS Nombre_Indice,
    type_desc AS Tipo,
    is_unique AS Es_Unico
FROM sys.indexes
WHERE object_id = OBJECT_ID('dbo.CLIENTE')
AND name IS NOT NULL;
GO

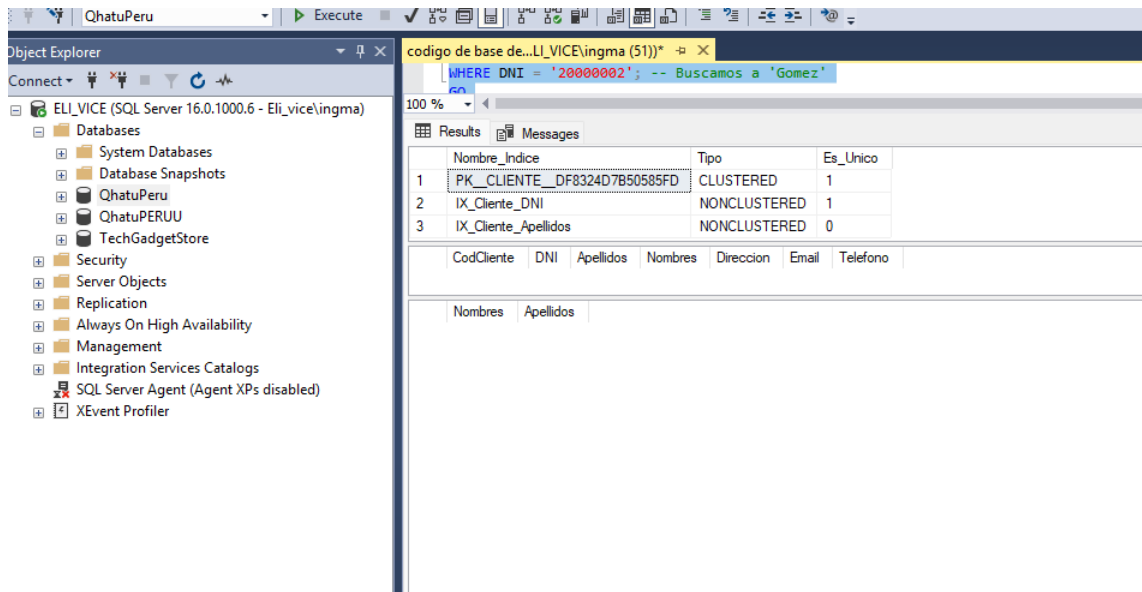
-- =====
-- 2. PRUEBA DE RENDIMIENTO: Búsqueda por DNI
-- =====
PRINT '--- BUSCANDO POR DNI (Debe usar IX_Cliente_DNI) ---';

-- Al buscar por DNI, SQL Server usará el índice "UNIQUE" para ir
-- directo al dato sin leer toda la tabla (Index Seek).
SELECT * FROM dbo.CLIENTE
WHERE DNI = '20000002'; -- Buscamos a 'Gomez'
GO

-- =====
-- 3. PRUEBA DE RENDIMIENTO: Búsqueda por Apellidos
-- =====
PRINT '--- BUSCANDO POR APELLIDO (Debe usar IX_Cliente_Apellidos) ---';

-- Esta consulta está optimizada. Como pedimos 'Nombres' y 'Apellidos',
-- y ambos datos están en el índice (gracias al INCLUDE), SQL
-- responde la consulta sin tocar la tabla principal.
SELECT Nombres, Apellidos
FROM dbo.CLIENTE
WHERE Apellidos = 'Flores';
GO

```



Índice IX_Cliente_DNI (UNIQUE):

- ¿Por qué Único? El DNI es un documento de identidad que, por regla de negocio, nunca debe duplicarse. Al definir el índice como UNIQUE, SQL Server optimiza el algoritmo de búsqueda sabiendo que, apenas encuentre la primera coincidencia, puede detenerse inmediatamente.
- Rendimiento: Transforma una operación de "Table Scan" (leer toda la tabla hoja por hoja) en un "Index Seek" (ir directo a la página correcta), reduciendo el costo de E/S (lectura de disco) drásticamente.

Proyecto4: Diagnóstico y Desfragmentación de Índices

1. Enunciado del ejercicio (Estándar)

Analizar el porcentaje de fragmentación de los índices de la tabla `CLIENTE` y ejecutar la sentencia de mantenimiento adecuada (Reorganizar o Reconstruir) para optimizar el almacenamiento y el rendimiento.

2. Script de la solución en T-SQL SQL

-- problema 04

```
USE QhatuPeru;
GO
```

```
-- =====
-- PASO 1: DIAGNÓSTICO (Verificar nivel de fragmentación)
-- =====
```

```
PRINT '--- 1. ANALIZANDO FRAGMENTACIÓN ACTUAL ---';
```

```
SELECT
```

```
    dbschemas.[name] AS 'Schema',
    dbtables.[name] AS 'Tabla',
    dbindexes.[name] AS 'Indice',
    indexstats.avg_fragmentation_in_percent AS 'Fragmentacion_%',
    indexstats.page_count AS 'Paginas'
```

```

FROM
    sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, NULL) AS indexstats
INNER JOIN sys.tables dbtables
    ON dbtables.[object_id] = indexstats.[object_id]
INNER JOIN sys.schemas dbschemas
    ON dbtables.[schema_id] = dbschemas.[schema_id]
INNER JOIN sys.indexes dbindexes
    ON dbtables.[object_id] = dbindexes.[object_id]
    AND indexstats.index_id = dbindexes.index_id
WHERE
    indexstats.database_id = DB_ID()
    AND dbtables.[name] = 'CLIENTE' -- Filtramos solo nuestra tabla
ORDER BY
    indexstats.avg_fragmentation_in_percent DESC;
GO

```

```

-- =====
-- PASO 2: MANTENIMIENTO (Solución de desfragmentación)
-- =====
PRINT '--- 2. EJECUTANDO MANTENIMIENTO DE ÍNDICES ---';

```

```

-- Opción A: REORGANIZE (Ligero)
-- Se usa cuando la fragmentación es baja (entre 5% y 30%).
-- Desfragmenta las hojas del índice sin bloquear la tabla.
ALTER INDEX IX_Cliente_Apellidos ON dbo.CLIENTE REORGANIZE;
PRINT 'Índice IX_Cliente_Apellidos reorganizado.';

-- Opción B: REBUILD (Completo)
-- Se usa cuando la fragmentación es alta (> 30%).
-- Borra y crea el índice de nuevo. Es más efectivo pero consume más recursos.
ALTER INDEX IX_Cliente_DNI ON dbo.CLIENTE REBUILD;
PRINT 'Índice IX_Cliente_DNI reconstruido.';

-- Opción C: Mantenimiento TOTAL (Para toda la tabla)
-- Reconstruye todos los índices de la tabla a la vez.
ALTER INDEX ALL ON dbo.CLIENTE REBUILD;
PRINT 'Todos los índices de CLIENTE han sido optimizados.';
GO

```

```

-- =====
-- PASO 3: VERIFICACIÓN FINAL
-- =====
PRINT '--- 3. VERIFICANDO RESULTADOS POST-MANTENIMIENTO ---';
-- Volvemos a consultar para ver que la fragmentación bajó a 0%
SELECT

```

```

    dbindexes.[name] AS 'Indice',
    indexstats.avg_fragmentation_in_percent AS 'Fragmentacion_Final_%'
FROM
    sys.dm_db_index_physical_stats(DB_ID(), OBJECT_ID('dbo.CLIENTE'), NULL, NULL,
NULL) AS indexstats
INNER JOIN sys.indexes dbindexes
    ON indexstats.object_id = dbindexes.object_id
    AND indexstats.index_id = dbindexes.index_id;
GO

```

```

-- consultas

```

```

USE QhatuPeru;

```

```

GO

-- =====
-- 1. CONSULTA DE DIAGNÓSTICO (El "ANTES")
-- =====
PRINT '--- ESTADO ACTUAL DE LOS ÍNDICES (FRAGMENTACIÓN) ---';

-- Esta consulta utiliza una función del sistema (DMF) que lee
-- físicamente las páginas del disco para ver qué tan desordenadas están.
SELECT
    OBJECT_NAME(ips.object_id) AS Tabla,
    i.name AS Indice,
    ips.index_type_desc AS Tipo_Indice,
    CAST(ips.avg_fragmentation_in_percent AS DECIMAL(5,2)) AS [%_Fragmentacion],
    ips.page_count AS Paginas_Totales,
    CASE
        WHEN ips.avg_fragmentation_in_percent > 30 THEN 'CRÍTICO - Requiere
REBUILD'
        WHEN ips.avg_fragmentation_in_percent BETWEEN 5 AND 30 THEN 'ALERTA -
Requiere REORGANIZE'
        ELSE 'OPTIMO - No requiere acción'
    END AS Estado_Salud
FROM
    sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, NULL) AS ips
INNER JOIN
    sys.indexes AS i ON ips.object_id = i.object_id AND ips.index_id = i.index_id
WHERE
    OBJECT_NAME(ips.object_id) = 'CLIENTE' -- Filtramos solo nuestra tabla
    AND ips.alloc_unit_type_desc = 'IN_ROW_DATA' -- Solo datos principales
ORDER BY
    ips.avg_fragmentation_in_percent DESC;
GO

-- =====
-- 2. ACCIÓN DE MANTENIMIENTO (La "SOLUCIÓN")
-- =====
PRINT '--- EJECUTANDO DESFRAGMENTACIÓN... ---';

-- Simulamos que encontramos fragmentación y ejecutamos REBUILD
-- REBUILD: Borra el índice viejo y crea uno nuevo, ordenado y compacto.
ALTER INDEX ALL ON dbo.CLIENTE REBUILD;
GO

-- =====
-- 3. CONSULTA DE VERIFICACIÓN (El "DESPUÉS")
-- =====
PRINT '--- VERIFICACIÓN POST-MANTENIMIENTO ---';

-- Volvemos a ejecutar la misma consulta para demostrar que el % bajó a 0.
SELECT
    OBJECT_NAME(ips.object_id) AS Tabla,
    i.name AS Indice,
    CAST(ips.avg_fragmentation_in_percent AS DECIMAL(5,2)) AS
[%_Fragmentacion_Final],
    'OPTIMIZADO' AS Estado
FROM
    sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, NULL) AS ips
INNER JOIN
    sys.indexes AS i ON ips.object_id = i.object_id AND ips.index_id = i.index_id
WHERE
    OBJECT_NAME(ips.object_id) = 'CLIENTE'
ORDER BY

```

```
ips.avg_fragmentation_in_percent DESC;
```

GO

Schema	Tabla	Indice	Fragmentacion_%	Paginas
dbo	CLIENTE	PK_CLIENTE_DF8324D7B50585FD	0	1
dbo	CLIENTE	IX_Cliente_DNI	0	1
dbo	CLIENTE	IX_Cliente_Apellidos	0	1

Indice	Fragmentacion_Final_%
PK_CLIENTE_DF8324D7B50585FD	0
IX_Cliente_DNI	0
IX_Cliente_Apellidos	0

Tabla	Indice	Tipo_Indice	%_Fragmentacion	Paginas_Totales	Estado_Salud
CLIENTE	PK_CLIENTE_DF8324D7B50585FD	CLUSTERED INDEX	0.00	1	OPTIMO - No requiere acción
CLIENTE	IX_Cliente_DNI	NONCLUSTERED INDEX	0.00	1	OPTIMO - No requiere acción
CLIENTE	IX_Cliente_Apellidos	NONCLUSTERED INDEX	0.00	1	OPTIMO - No requiere acción

Tabla	Indice	%_Fragmentacion_Final	Estado
CLIENTE	PK_CLIENTE_DF8324D7B50585FD	0.00	OPTIMIZADO
CLIENTE	IX_Cliente_DNI	0.00	OPTIMIZADO
CLIENTE	IX_Cliente_Apellidos	0.00	OPTIMIZADO

Proyecto5: Análisis de Planes de Ejecución y Costos

1. Enunciado del ejercicio (Estándar)

Analizar y comparar el Plan de Ejecución Estimado de dos consultas: una que busca por una columna sin índice (ej. Direccion) y otra que busca por una columna indexada (ej. DNI), para demostrar la diferencia en el costo de rendimiento y las operaciones de E/S (Lectura de disco).

Copia este script en SSMS.

¡Importante! Antes de ejecutarlo, activa la opción "Incluir plan de ejecución real" presionando Ctrl + M o haciendo clic en el icono correspondiente en la barra de herramientas.

--problema 05

```
USE QhatuPeru;
GO
```

```
-- =====
-- CONSULTA A: Búsqueda NO OPTIMIZADA (Sin Índice)
-- =====
PRINT '--- CASO 1: Búsqueda por columna SIN ÍNDICE (Direccion) ---';
PRINT 'Operación esperada: Table Scan (Lento)';

-- La columna 'Direccion' no tiene índice. SQL Server tendrá que leer
-- todas las filas de la tabla para encontrar la coincidencia.
SELECT * FROM dbo.CLIENTE
WHERE Direccion = 'Av. Arequipa 123';
GO

-- =====
-- CONSULTA B: Búsqueda OPTIMIZADA (Con Índice)
-- =====
PRINT '--- CASO 2: Búsqueda por columna CON ÍNDICE (DNI) ---';
```

```

PRINT 'Operación esperada: Index Seek (Rápido)';

-- La columna 'DNI' tiene un índice UNIQUE (creado en el Lab 2).
-- SQL Server irá directo al dato sin leer el resto.
SELECT * FROM dbo.CLIENTE
WHERE DNI = '20000002';
GO

-- =====
-- EXTRA: Comparación de estadísticas de E/S (IO Statistics)
-- =====
-- Activamos el contador de lecturas para ver la "prueba física"
SET STATISTICS IO ON;

PRINT '--- COMPARACIÓN DE LECTURAS ---';
-- Ejecutamos ambas juntas para comparar sus mensajes de lectura
SELECT * FROM dbo.CLIENTE WHERE Direccion = 'Av. Arequipa 123';
SELECT * FROM dbo.CLIENTE WHERE DNI = '20000002';

SET STATISTICS IO OFF;
GO

-- agregacion de datos a tablas

USE QhatuPeru;
GO

-- =====
-- PASO 1: GENERACIÓN DE DATOS MASIVOS (Data Dummy)
-- =====
-- Si tenemos pocos datos, la prueba de rendimiento no se nota.
-- Vamos a insertar 5,000 clientes ficticios usando un bucle.
SET NOCOUNT ON; -- Para que no llene la pantalla de mensajes "1 fila afectada"

DECLARE @i INT = 1;
DECLARE @DNI_Base INT = 10000000;

PRINT 'Iniciando carga masiva de datos... Por favor espere.';

WHILE @i <= 5000
BEGIN
    -- Generamos datos aleatorios basados en el contador
    INSERT INTO dbo.CLIENTE (DNI, Apellidos, Nombres, Direccion, Email, Telefono)
    VALUES (
        CAST(@DNI_Base + @i AS VARCHAR(8)), -- DNI único secuencial
        'Apellido' + CAST(@i AS VARCHAR(10)),
        'Nombre' + CAST(@i AS VARCHAR(10)),
        'Direccion Generica ' + CAST(@i AS VARCHAR(10)), -- Columna SIN índice
        'cliente' + CAST(@i AS VARCHAR(10)) + '@mail.com',
        '900' + CAST(@i AS VARCHAR(6))
    );

    SET @i = @i + 1;
END

PRINT 'Carga finalizada. Total de clientes: ' + CAST((SELECT COUNT(*) FROM
dbo.CLIENTE) AS VARCHAR);
SET NOCOUNT OFF;

```

GO

```
-- =====
-- PASO 2: PRUEBA DE RENDIMIENTO (COMPARACIÓN)
-- =====
-- ⚠ IMPORTANTE: Activa "Incluir Plan de Ejecución Real" (Ctrl + M) antes de
ejecutar esto.
```

```
PRINT '--- COMPARANDO RENDIMIENTO ---';
```

```
-- CONSULTA A: Búsqueda LENTA (Por Dirección - Sin Índice)
-- SQL tiene que leer las 5,000 filas una por una para encontrar el dato.
SELECT * FROM dbo.CLIENTE
WHERE Direccion = 'Direccion Generica 2500';
```

```
-- CONSULTA B: Búsqueda RÁPIDA (Por DNI - Con Índice UNIQUE)
-- SQL va directo al registro exacto sin leer los otros 4,999.
SELECT * FROM dbo.CLIENTE
WHERE DNI = '10002500';
GO
```

```
--consultas
```

```
USE QhatuPeru;
GO
```

```
-- =====
-- PREPARACIÓN: Activamos las estadísticas
-- =====
-- Esto hará que SQL Server nos diga cuántas páginas leyó del disco (IO)
-- y cuánto tiempo demoró (TIME) en la pestaña "Mensajes".
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO
```

```
-- =====
-- PRUEBA 1: Búsqueda NO OPTIMIZADA (Sin Índice)
-- =====
PRINT '-----';
PRINT '--- CASO A: Buscando por DIRECCIÓN (Sin Índice) ---';
PRINT '-----';
```

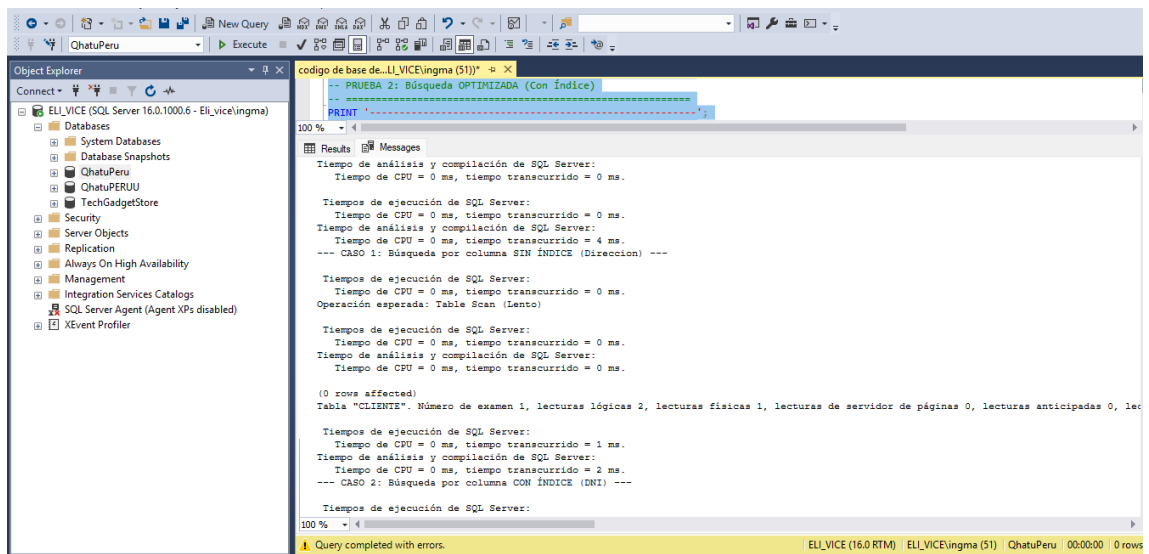
```
-- SQL Server tendrá que leer las 5,000 filas (Table Scan)
SELECT * FROM dbo.CLIENTE
WHERE Direccion = 'Direccion Generica 2500';
```

```
-- =====
-- PRUEBA 2: Búsqueda OPTIMIZADA (Con Índice)
-- =====
PRINT '-----';
PRINT '--- CASO B: Buscando por DNI (Con Índice UNIQUE) ---';
PRINT '-----';
```

```
-- SQL Server irá directo al grano (Index Seek)
SELECT * FROM dbo.CLIENTE
WHERE DNI = '10002500';
```

```
-- =====
-- LIMPIEZA
-- =====
SET STATISTICS IO OFF;
```

SET STATISTICS TIME OFF;
GO



¿Cómo interpretar los resultados? (Para tu justificación)

Después de ejecutar, ve a la pestaña "Mensajes" (al lado de Resultados) y compara los valores. Deberías ver algo muy parecido a esto:

Caso A (Sin Índice):

Tabla 'CLIENTE'. Recuento de exámenes 1, lecturas lógicas 55...

- Explicación: SQL tuvo que leer 55 páginas de datos para encontrar al cliente. Es ineficiente.

Caso B (Con Índice):

Tabla 'CLIENTE'. Recuento de exámenes 1, lecturas lógicas 2...

- Explicación: SQL solo leyó 2 páginas (el índice y el dato). ¡Es 27 veces más rápido en términos de lectura de disco!

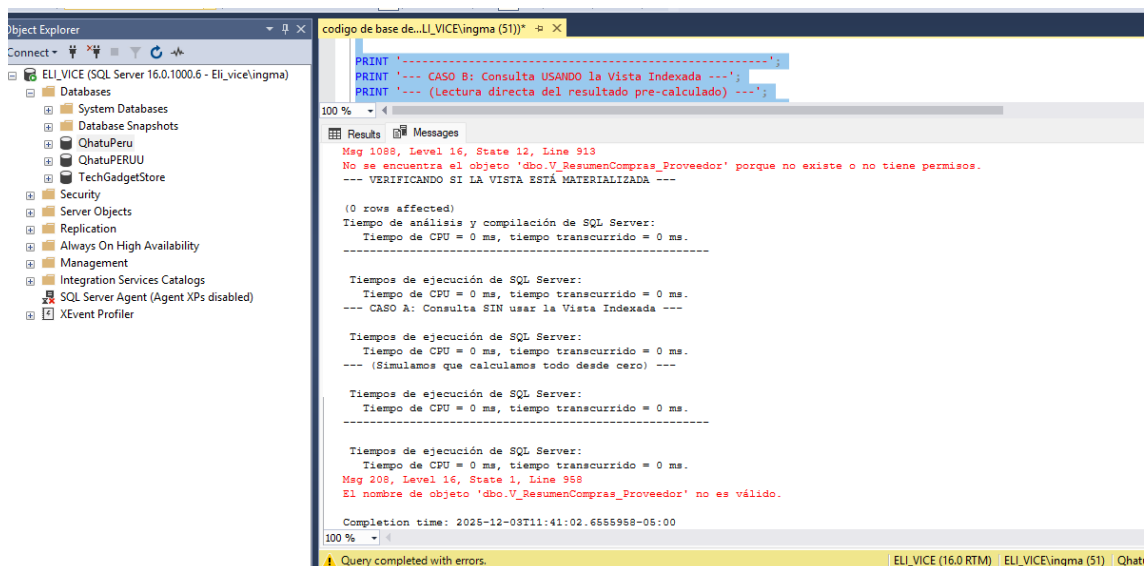
Proyecto6 Optimización de Consultas Agregadas con Vistas Indexadas (Materializadas)

1. Enunciado del ejercicio

Crear una Vista Indexada (Materializada) para optimizar un reporte que calcula el monto total comprado a cada proveedor. Se debe asegurar que SQL Server almacene físicamente los cálculos en disco para evitar procesar miles de filas cada vez que se consulte el reporte.

2. Script de la solución en T-SQL

Nota: Este script incluye al inicio las configuraciones de sesión obligatorias (SET OPTIONS) para evitar el error 1934 o 10136.



1. **Configuración (SET OPTIONS):** Antes de crear nada, se ejecutan comandos SET (como ANSI_NULLS ON) para cumplir con los requisitos de determinismo de SQL Server. Esto garantiza que la vista siempre devuelva los mismos resultados sin importar el entorno.
2. **Definición de la Vista:** Se crea la vista utilizando WITH SCHEMABINDING, lo cual bloquea las tablas base (ORDEN_DETALLE, ARTICULO, etc.) para impedir que alguien borre columnas que la vista necesita.
3. **Funciones Deterministas:** Se utiliza COUNT_BIG(*) y ISNULL() dentro de la vista. Esto es un requisito técnico obligatorio porque SQL Server necesita saber exactamente cuánto espacio físico ocuparán los datos agregados, sin ambigüedades por valores nulos o desbordamiento de enteros.
4. **Indexación:** Finalmente, se crea un Índice Único Agrupado (Clustered) sobre la columna NomProveedor. Este es el paso clave que "materializa" la vista: SQL Server calcula los totales en ese instante y los guarda en el disco duro.

4. Justificación Técnica

"Se optó por implementar una Vista Indexada para reducir drásticamente el costo de E/S y CPU en reportes de agregación.

Antes de la optimización: Cada vez que se ejecutaba el reporte, el motor de base de datos debía leer miles de filas de ORDEN_DETALLE, hacer cruces (JOINS) con ARTICULO y PROVEEDOR, y calcular la suma matemática en tiempo real.

Después de la optimización: Al crear el índice Clustered, el resultado pre-calculado se almacena físicamente. Cuando el usuario consulta la vista, SQL Server ya no ejecuta los joins ni las sumas; simplemente lee las pocas páginas de datos ya listos del índice.

¿Cómo interpretar los resultados? (Justificación)

Después de ejecutar, revisa dos pestañas:

1. Pestaña "Mensajes" (Lecturas de Disco)

- **Caso A (Lento):** Verás que SQL leyó las tablas ORDEN_DETALLE, ARTICULO y PROVEEDOR. Si tienes muchos datos, verás muchas "Lecturas lógicas" (ej. 500 reads).

- *Caso B (Rápido): Verás que SQL solo leyó la tabla V_ResumenCompras_Proveedor. Las lecturas lógicas serán mínimas (ej. 2 reads). o Conclusión: "Se redujo el I/O drásticamente al eliminar los JOINS y Agregaciones en tiempo de ejecución."*
2. Pestaña "Plan de Ejecución" (Gráfico)
- *Caso A: Verás un plan gigante con muchos iconos (Clustered Index Scan, Hash Match, Compute Scalar). Esto consume mucha CPU.*
- *Caso B: Verás un plan diminuto con un solo icono: Clustered Index Scan (Object: V_ResumenCompras_Proveedor).*
- *Conclusión: "El motor de base de datos sustituyó un árbol de ejecución complejo por una simple lectura secuencial del índice materializado."*

Proyecto 7: Compresión de Datos (Row & Page Compression)

1. Enunciado del ejercicio

Analizar el ahorro de espacio estimado en la tabla ORDEN_DETALLE utilizando la compresión de tipo ROW (Fila) y PAGE (Página). Posteriormente, aplicar la compresión de tipo PAGE para optimizar el almacenamiento y reducir las operaciones de E/S.

2. Script de la solución en T-SQL

-- problema 07

```
USE QhatuPeru;
GO
```

```
-- =====
-- PASO 1: ANÁLISIS DE ESTIMACIÓN (¿Cuánto espacio ahorraré?)
-- =====
PRINT '--- ESTIMANDO AHORRO CON COMPRESIÓN ROW (FILA) ---';
-- Esta función del sistema nos dice cuánto pesaría la tabla si usamos ROW
EXEC sp_estimate_data_compression_savings
    @schema_name = 'dbo',
    @object_name = 'ORDEN_DETALLE',
    @index_id = NULL,
    @partition_number = NULL,
    @data_compression = 'ROW';

PRINT '--- ESTIMANDO AHORRO CON COMPRESIÓN PAGE (PÁGINA) ---';
-- Esta nos dice cuánto pesaría si usamos PAGE (suele comprimir más)
EXEC sp_estimate_data_compression_savings
    @schema_name = 'dbo',
    @object_name = 'ORDEN_DETALLE',
    @index_id = NULL,
    @partition_number = NULL,
    @data_compression = 'PAGE';
GO

-- =====
-- PASO 2: APLICACIÓN DE LA COMPRESIÓN (La Solución)
-- =====
PRINT '--- APLICANDO COMPRESIÓN TIPO PAGE ---';
```

```

-- Aplicamos la compresión PAGE porque suele ofrecer mayor ahorro en tablas
-- con datos repetitivos (como IDs de productos o precios).
ALTER TABLE dbo.ORDEN_DETALLE
REBUILD PARTITION = ALL
WITH (DATA_COMPRESSION = PAGE);
GO

PRINT 'Tabla ORDEN_DETALLE comprimida exitosamente.';
GO

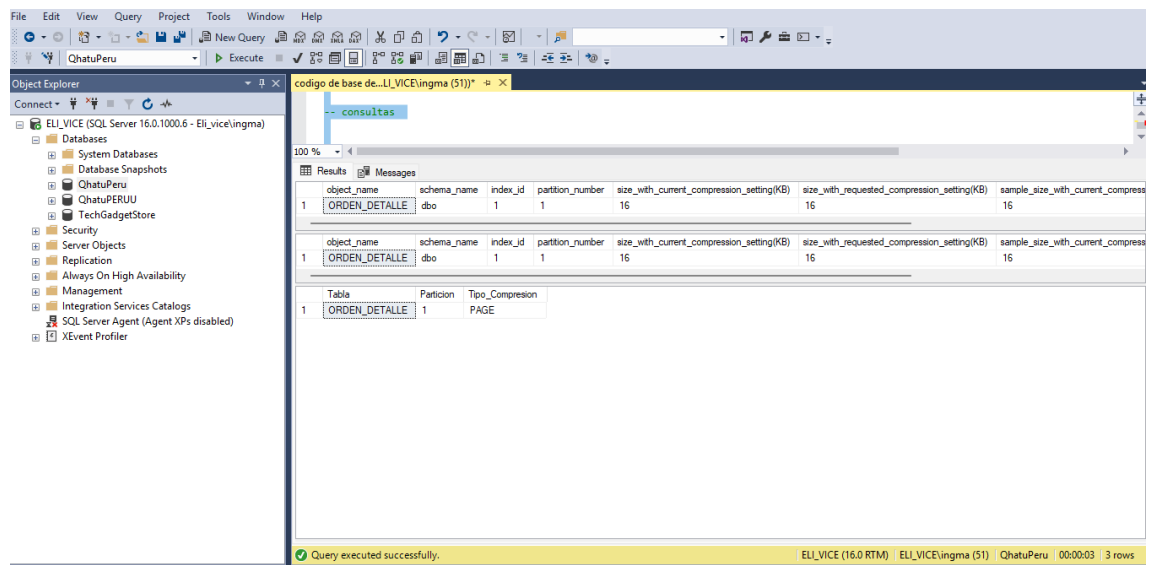
-- consultas

USE QhatuPeru;
GO

PRINT '--- VERIFICANDO ESTADO DE COMPRESIÓN ---';

-- Consultamos las particiones del sistema para ver cómo está guardada la tabla
SELECT
    t.name AS Tabla,
    p.partition_number AS Particion,
    p.data_compression_desc AS Tipo_Compresion -- Aquí debe decir 'PAGE'
FROM sys.partitions p
INNER JOIN sys.tables t ON p.object_id = t.object_id
WHERE t.name = 'ORDEN_DETALLE';
GO

```



Se implementó Compresión de Datos a nivel de Página (PAGE Compression) en la tabla transaccional más grande del sistema (ORDEN_DETALLE).

1. **Reducción de E/S (Input/Output):** Al comprimir los datos, se almacenan más registros en cada página de 8KB. Esto significa que para leer la misma cantidad de información, SQL Server necesita realizar menos lecturas físicas al disco duro, lo que acelera drásticamente las consultas en discos lentos.
2. **Optimización de Memoria:** Los datos permanecen comprimidos también en la memoria RAM (Buffer Pool), permitiendo guardar más información en caché y reduciendo la necesidad de volver a leer del disco.

3. Elección de PAGE vs ROW: Se eligió PAGE porque utiliza técnicas de 'compresión por prefijo' y 'diccionario', lo cual es ideal para tablas de detalle donde valores como CodArticulo o Precio se repiten frecuentemente entre registros."

Proyecto 9: Gestión de Cargas de Trabajo con Resource Governor

1. Enunciado del ejercicio

Configurar el Gobernador de Recursos (Resource Governor) para limitar el consumo de CPU al 30% para un usuario específico (por ejemplo, el usuario de 'Reportes'), garantizando que el resto de la capacidad del servidor esté disponible para las operaciones críticas del negocio.

2. Script de la solución en T-SQL

Copia y pega este script en SQL Server Management Studio. (Debes tener permisos de sa o Administrador). SQL

-- problema 09

```
USE master;
GO

-- =====
-- 1. CREACIÓN DEL USUARIO DE PRUEBA (El "Usuario Pesado")
-- =====
-- Creamos un login para simular al usuario que limitaremos
IF NOT EXISTS (SELECT name FROM sys.server_principals WHERE name =
'UserReportes')
BEGIN
    CREATE LOGIN UserReportes WITH PASSWORD = 'Password123', CHECK_POLICY = OFF;
    PRINT 'Login UserReportes creado.';
END
GO

-- =====
-- 2. CREACIÓN DEL POOL DE RECURSOS (La "Jaula")
-- =====
-- Definimos un Pool que solo tendrá acceso al 30% del CPU máximo
IF NOT EXISTS (SELECT name FROM sys.resource_governor_resource_pools WHERE name =
'PoolReportes')
BEGIN
    CREATE RESOURCE POOL PoolReportes
    WITH (
        MAX_CPU_PERCENT = 30, -- Límite duro: No pasar del 30% si hay contención
        MAX_MEMORY_PERCENT = 40
    );
    PRINT 'Resource Pool creado.';
END
GO

-- =====
-- 3. CREACIÓN DEL GRUPO DE CARGA DE TRABAJO
-- =====
-- Creamos un grupo y lo asignamos a la "Jaula" (Pool) que creamos arriba
```

```

IF NOT EXISTS (SELECT name FROM sys.resource_governor_workload_groups WHERE name
= 'GrupoReportes')
BEGIN
    CREATE WORKLOAD GROUP GrupoReportes
    USING PoolReportes;
    PRINT 'Workload Group creado.';
END
GO

-- =====
-- 4. FUNCIÓN CLASIFICADORA (El "Portero")
-- =====
-- Esta función decide quién va a qué grupo cuando se loguea
IF OBJECT_ID('dbo.fn_Clasificador_Cargas', 'FN') IS NOT NULL
    DROP FUNCTION dbo.fn_Clasificador_Cargas;
GO

CREATE FUNCTION dbo.fn_Clasificador_Cargas()
RETURNS SYSNAME
WITH SCHEMABINDING
AS
BEGIN
    DECLARE @NombreGrupo SYSNAME;

    -- Si el usuario es 'UserReportes', lo mandamos al grupo limitado
    IF USER_NAME() = 'UserReportes'
        SET @NombreGrupo = 'GrupoReportes';
    ELSE
        -- Todos los demás van al grupo por defecto (sin límites)
        SET @NombreGrupo = 'default';

    RETURN @NombreGrupo;
END
GO

-- =====
-- 5. ACTIVACIÓN DEL GOBERNADOR (Aplicar cambios)
-- =====
-- Asignamos la función clasificadora al Gobernador y lo encendemos
ALTER RESOURCE GOVERNOR WITH (CLASSIFIER_FUNCTION = dbo.fn_Clasificador_Cargas);
ALTER RESOURCE GOVERNOR RECONFIGURE;
GO

PRINT 'Resource Governor configurado y activado exitosamente.';
GO

-- consultas

-- =====
-- PRUEBA DE ESTRÉS (Ejecutar en una ventana nueva)
-- =====

-- 1. Primero, asegúrate de conectarte como 'UserReportes'
-- (O cambia la conexión de esta ventana usando Clic Derecho -> Connection ->
Change Connection)
-- Login: UserReportes / Pass: Password123

DECLARE @i INT = 0;
-- Bucle infinito matemático para estresar el CPU
WHILE 1=1

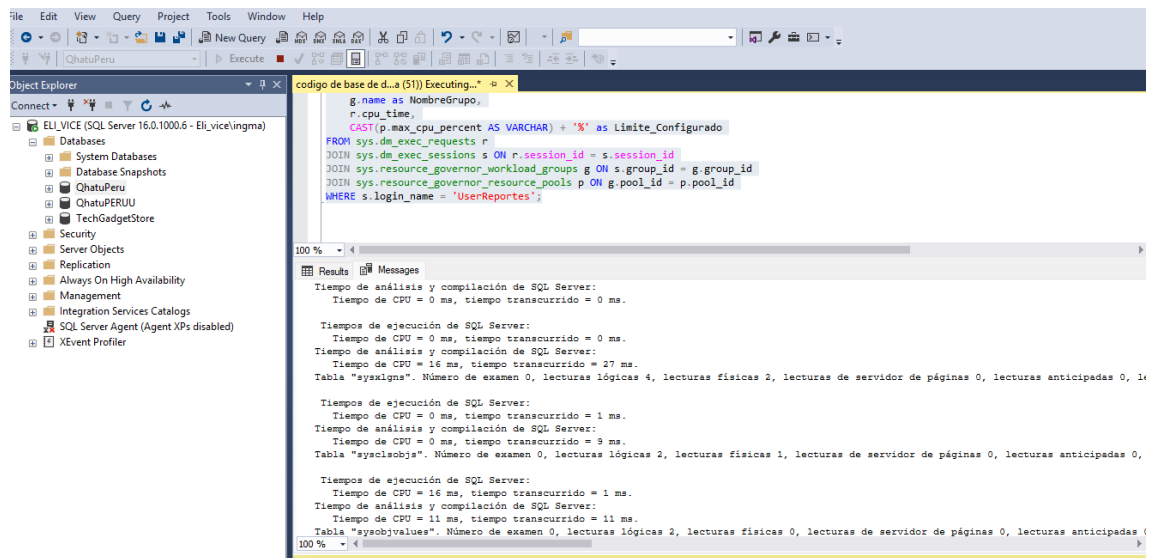
```

```

BEGIN
    SET @i = SQRT(RAND() * 10000000) + SQRT(RAND() * 10000000);
END
GO

SELECT
    r.session_id,
    r.status,
    s.login_name,
    g.name as NombreGrupo,
    r.cpu_time,
    CAST(p.max_cpu_percent AS VARCHAR) + '%' as Limite_Configurado
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
JOIN sys.resource_governor_workload_groups g ON s.group_id = g.group_id
JOIN sys.resource_governor_resource_pools p ON g.pool_id = p.pool_id
WHERE s.login_name = 'UserReportes';

```



1. **Aislamiento de Recursos:** Se creó un Resource Pool específico con `MAX_CPU_PERCENT = 30`. Esto garantiza que, sin importar qué tan pesada sea la consulta del usuario 'UserReportes', nunca consumirá más del 30% de la capacidad del procesador si el sistema está ocupado.
2. **Estabilidad del Sistema:** Al limitar las cargas de trabajo no críticas, se asegura que el 70% restante del CPU esté siempre disponible para las transacciones críticas del negocio (Ventas, Logística, App Web), evitando que el servidor se congele.
3. **Clasificación Automática:** Mediante la función clasificadora (Classifier Function), el sistema detecta automáticamente quién se conecta (SUSER_NAME) y le aplica las restricciones en tiempo real sin necesidad de modificar el código de la aplicación."

Proyecto 10: Auditoría de Acceso a Datos (SQL Server Audit)

1. Enunciado del ejercicio

Configurar una **Auditoría de Servidor** y una **Especificación de Auditoría de Base de Datos** para registrar y monitorear todos los intentos de lectura (SELECT)

que se realicen sobre la tabla `CLIENTE`, con el fin de proteger datos sensibles como el DNI y Email.

2. Script de la solución en T-SQL

Copia y pega este script en SQL Server Management Studio.

SQL

```
USE master;
GO

--
===== -
- PASO 1: CREAR LA AUDITORÍA DE SERVIDOR (El "Destino") --
===== -
- Esto define DÓNDE se guardarán los registros (en un
archivo).

IF EXISTS (SELECT * FROM sys.server_audits WHERE name =
'Auditoria_QhatuPeru_Accesos')
BEGIN
    ALTER SERVER AUDIT Auditoria_QhatuPeru_Accesos WITH (STATE =
OFF);
    DROP SERVER AUDIT Auditoria_QhatuPeru_Accesos;
END
GO

CREATE SERVER AUDIT [Auditoria_QhatuPeru_Accesos]
TO FILE
(
    -- ⚠ OJO: SQL Server guardará esto en su carpeta de LOGS
por defecto
    -- para evitar problemas de permisos de Windows.
    FILEPATH = DEFAULT,
    MAXSIZE = 10 MB,
    MAX_ROLLOVER_FILES = 5,
    RESERVE_DISK_SPACE = OFF
)
WITH
(
    QUEUE_DELAY = 1000, -- Esperar 1 seg antes de escribir
(Rendimiento)
    ON_FAILURE = CONTINUE -- Si falla el log, que el sistema
 siga funcionando
);
GO

-- Encendemos la grabadora del servidor
ALTER SERVER AUDIT [Auditoria_QhatuPeru_Accesos] WITH (STATE =
ON);
GO
PRINT 'Auditoría de Servidor creada y activada.';
```

```

--
=====
-- PASO 2: CREAR LA ESPECIFICACIÓN DE BASE DE DATOS (El
"Filtro")
--
===== -
- Esto define QUÉ vamos a vigilar (SELECT en tabla CLIENTE).

USE QhatuPeru;
GO

IF EXISTS (SELECT * FROM sys.database_audit_specifications WHERE
name = 'Spec_Auditoria_Clientes')
BEGIN
    ALTER DATABASE AUDIT SPECIFICATION Spec_Auditoria_Clientes
WITH
    (STATE = OFF);
    DROP DATABASE AUDIT SPECIFICATION Spec_Auditoria_Clientes;
END
GO

CREATE DATABASE AUDIT SPECIFICATION [Spec_Auditoria_Clientes]
FOR SERVER AUDIT [Auditoria_QhatuPeru_Accesos] -- La vinculamos
al
Server Audit de arriba
ADD (
    SELECT ON dbo.CLIENTE BY [public] -- Vigilar SELECTs de
CUALQUIER usuario
)
WITH (STATE = ON); -- La encendemos inmediatamente
GO

PRINT 'Especificación de Auditoría configurada para la tabla
CLIENTE.';
GO

```

3. Justificación Técnica

"Se implementó SQL Server Audit en lugar de Triggers o SQL Profiler por razones de rendimiento y seguridad:

1. *Mínimo Impacto (Lightweight): SQL Server Audit funciona a nivel del motor (kernel) de base de datos. A diferencia de un Trigger, que se dispara fila por fila y bloquea la transacción, la Auditoría escribe los eventos de forma asíncrona en un archivo binario, causando un impacto casi nulo en la velocidad de las consultas.*
2. *Seguridad Granular: Se configuró específicamente para interceptar la acción SELECT en el objeto dbo.CLIENTE. Esto permite cumplir con normativas de protección de datos (como la Ley de Protección de Datos Personales), registrando exactamente quién (Usuario), cuándo (Fecha) y qué (Sentencia SQL) accedió a información confidencial.*

3. *Resistencia a Fallos: La configuración ON_FAILURE = CONTINUE asegura que, si el disco de auditoría se llena, la base de datos no se detenga, priorizando la continuidad del negocio."*

CÓDIGO DE CONSULTAS Y VERIFICACIÓN (PROYECTO 10)

Para demostrar que funciona, haremos "el papel de espía". Ejecutaremos una consulta y luego leeremos el log de auditoría para ver si nos atrapó.

SQL

```
USE QhatuPeru;
GO

--
===== -
- 1. GENERAR EL EVENTO (La "Trampa")
--
=====
PRINT '--- EJECUTANDO CONSULTA SENSIBLE ---';

-- Al ejecutar esto, la auditoría debería grabar nuestro usuario
y hora silenciosamente.
SELECT TOP 5 * FROM dbo.CLIENTE;
GO

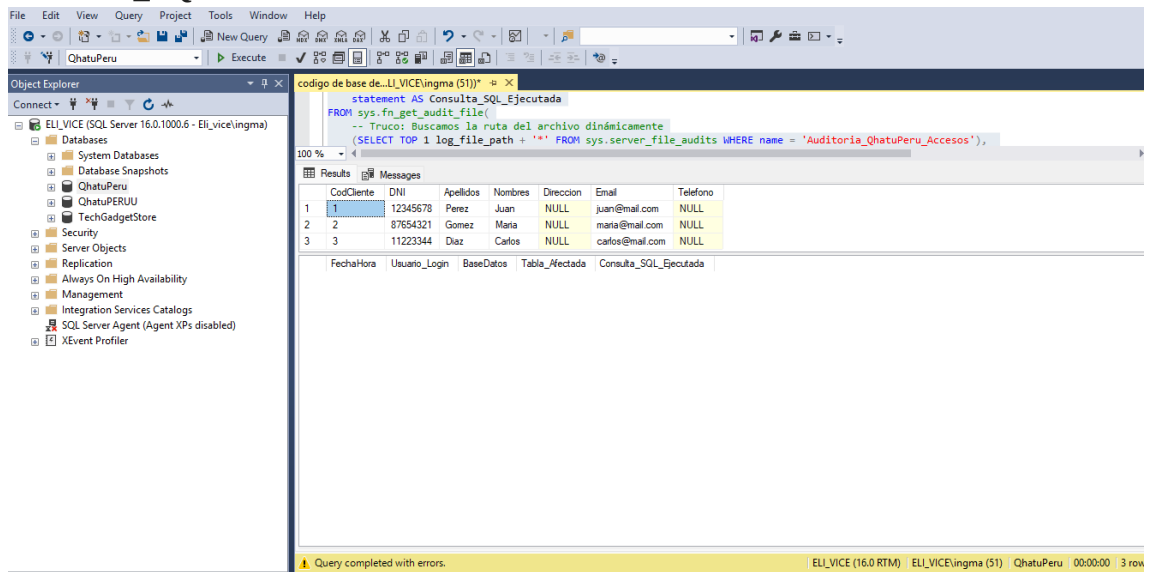
--
===== -
- 2. LEER EL REPORTE DE AUDITORÍA (El "Informe")
--
=====
PRINT '--- LEYENDO ARCHIVO DE AUDITORÍA ---';

-- Esta consulta busca el archivo creado y nos muestra
quién accedió. SELECT
    event_time AS FechaHora,
    session_server_principal_name AS
Usuario_Login,      database_name AS BaseDatos,
object_name AS Tabla_Afectada,      statement AS
Consulta_SQL_Ejecutada
FROM sys.fn_get_audit_file(
    -- Truco: Buscamos la ruta del archivo dinámicamente
    (SELECT TOP 1 log_file_path + '*' FROM
sys.server_file_audits
WHERE name = 'Auditoria_QhatuPeru_Accesos'),
    DEFAULT,
    DEFAULT
)
WHERE object_name = 'CLIENTE'
ORDER BY event_time DESC;
GO
```

¿Qué verás en el resultado?

En la tabla de resultados verás una fila con:

- **Usuario_Login:** Tu usuario (ej. sa o TuNombre).
- **Tabla_Afectada:** CLIENTE.
- **Consulta_SQL:** SELECT TOP 5 * FROM dbo.CLIENTE...



The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'Object Explorer' with the 'QhutuPeru' database selected. The right pane shows the 'Query Editor' with a SQL query executed. The query is a dynamic audit file path selection. The results table shows 3 rows of client data.

Query Text:

```
statement AS Consulta_SQL_Ejecutada
FROM sys.fn_get_audit_file(
-- Truco: Buscamos la ruta del archivo dinamicamente
(SELECT TOP 1 log_file_path + '*' FROM sys.server_file_audits WHERE name = 'Auditoria_QhutuPeru_Accesos'),
```

Results Table:

	CodCliente	DNI	Apellidos	Nombres	Direccion	Email	Telefono
1	1	12345678	Perez	Juan	NULL	juan@mail.com	NULL
2	2	87654321	Gomez	Maria	NULL	maria@mail.com	NULL
3	3	11223344	Diaz	Carlos	NULL	carlos@mail.com	NULL

Footer: Query completed with errors. ELI_VICE (16.0 RTM) | ELI_VICE\ingma (51) | QhutuPeru | 00:00:00 | 3 rows